Thermodynamic State Vector Interface & Exergy Tracking Architecture: Establishing First and Second Law Compliance in Planetary Biosphere Simulations

Lead Scientific Communications & Academic Outreach Agent
Web of Life Project
`https://github.com/pascalranoroarijaona/WebOfLife`

Sprint 019

Abstract

Simulating planetary ecosystems and biosphere-atmosphere interactions under high-fidelity biophysical constraints requires rigorous enforcement of thermodynamic principles. While conservation of mass and energy (First Law) is standard in biogeochemical modeling, tracking thermodynamic irreversibilities, entropy generation ($\dot{S}_{\text{gen}}$), and exergy destruction ($\dot{I}$) is essential for modeling ecosystem thermodynamic efficiency and metabolic degradation. This paper details the implementation of Sprint 019, which introduces the **Thermodynamic State Vector Interface** (`src/thermodynamics/types.ts`) within the Web of Life framework. We formalize TypeScript interfaces and abstract base classes that enforce the Clausius-Duhem inequality ($\dot{S}_{\text{gen}} \geq 0$) and execute Gouy-Stodola exergy destruction tracking ($\dot{I} = T_0 \dot{S}_{\text{gen}}$) across all monad process transitions under solar-only energetic forcing.

1 Introduction and Systems Ecology Context

As the Web of Life simulation evolves into a robust, high-fidelity biophysical planetary model (Gaia Pod architecture), managing heat, work, matter conservation, and irreversibilities requires an absolute thermodynamic standard. Previous sprints established baseline biogeochemical cycling (`src/cycles/`), thermodynamic monad processes (`src/thermodynamics/thermodynamic_monad_process`) and structural property calculations.

Sprint 019 establishes the **Thermodynamic State Vector Interface** in `src/thermodynamics/types.ts`. This specification sets formal TypeScript interfaces and classes for:

1. Internal entropy generation rates ($\dot{S}_{\text{gen}} \geq 0$).
2. Exergy destruction rates ($\dot{I} = T_0 \dot{S}_{\text{gen}}$ where T_0 is the ambient dead-state temperature).
3. Boundary heat and mass flux vector arrays (`BoundaryFluxVector`).
4. Strict Monad stock transitions preserving mass and energy under solar-only energetic forcing.

2 Thermodynamic First & Second Law Compliance

2.1 First Law: Energy Conservation

For any control volume V bounded by surface ∂V, the total energy change E_{system} is governed by:

$$\frac{dE_{\text{system}}}{dt} = \sum_k \dot{Q}_k - \dot{W}_{\text{useful}} + \sum_i \dot{m}_i h_i \tag{1}$$

Within our Earth Pod simulation:

- **Solar Input Only:** External work inputs $\dot{W}$ are restricted to radiative solar input (Φ_{solar}) and outgoing longwave thermal radiation. No internal hidden energy generation sources exist.
- **Matter Conservation:** All elemental stocks (Carbon, Nitrogen, Phosphorus, Water) tracked via monad processes must balance identically across state transitions: $\sum \Delta\text{Stock} = 0$ modulo boundary mass fluxes.

2.2 Second Law: Entropy Generation & Exergy Destruction

The rate of entropy change in the system is given by the Gouy-Stodola theorem extension:

$$\frac{dS_{\text{system}}}{dt} = \sum_k \frac{\dot{Q}_k}{T_k} + \sum_i \dot{m}_i s_i + \dot{S}_{\text{gen}} \tag{2}$$

Where internal entropy generation $\dot{S}_{\text{gen}}$ must satisfy the Clausius-Duhem inequality:

$$\dot{S}_{\text{gen}} \geq 0 \tag{3}$$

The **Exergy Destruction Rate** ($\dot{I}$), representing lost work potential due to irreversible processes (such as metabolic respiration, chemical dissipation, and thermal conduction), is defined as:

$$\dot{I} = T_0 \dot{S}_{\text{gen}} \tag{4}$$

where $T_0 = 288.15$ K is the standard planetary dead-state reference temperature.

3 Class Hierarchy Additions & Interface Contracts

The core interfaces and abstract classes defined in `src/thermodynamics/types.ts` enforce these thermodynamic bounds at runtime.

3.1 Boundary Flux Vector and State Vector

```
export interface BoundaryFluxVector {
  solarRadiationFlux: number;       // W/m^2 (>= 0)
  thermalRadiationFlux: number;     // W/m^2 (<= 0 during net emission)
  sensibleHeatFlux: number;         // W/m^2
  latentHeatFlux: number;           // W/m^2
  massFluxes: Map<string, number>;  // kg/s or mol/s per species
}

export interface ThermodynamicStateVector {
  internalEnergy: number;           // J
  entropy: number;                  // J/K
  temperature: number;              // K
  ambientTemperature: number;       // K (Default T_0 = 288.15)
  entropyGenerationRate: number;    // W/K, must be >= 0
  exergyDestructionRate: number;    // W, I_dot = T_0 * S_dot_gen
  exergy: number;                   // J
  boundaryFluxes: BoundaryFluxVector;
}
```

3.2 Abstract Base Class Enforcement

```
import { IThermodynamicProcessMonad, ThermodynamicStateVector,
         ThermodynamicDerivativeResult } from './types';

export abstract class BaseThermodynamicProcessMonad
  implements IThermodynamicProcessMonad {
  abstract readonly processId: string;

  abstract evaluate(state: ThermodynamicStateVector, dt: number):
    ThermodynamicDerivativeResult;

  public transit(state: ThermodynamicStateVector, dt: number):
    ThermodynamicStateVector {
    const deriv = this.evaluate(state, dt);

    if (deriv.entropyGenerationRate < 0) {
      throw new Error(
        `Second Law Violation in monad '${this.processId}': ` +
        `S_gen_dot = ${deriv.entropyGenerationRate} W/K < 0.`
      );
    }

    const T_0 = state.ambientTemperature;
    const computedExergyDestruction = T_0 * deriv.entropyGenerationRate;

    return {
      internalEnergy: state.internalEnergy + deriv.dInternalEnergy,
      entropy: state.entropy + deriv.dEntropy,
      temperature: state.temperature,
      ambientTemperature: T_0,
      entropyGenerationRate: deriv.entropyGenerationRate,
      exergyDestructionRate: computedExergyDestruction,
      exergy: Math.max(0, state.exergy - computedExergyDestruction * dt),
      boundaryFluxes: state.boundaryFluxes
    };
  }
}
```

4 Verification and Test Suite Plan

To ensure robustness, `tests/sprint_019.test.ts` validates:

1. **Second Law Enforcement:** Mock monads returning $\dot{S}_{\text{gen}} < 0$ throw immediate runtime exceptions.
2. **Exergy Scaling:** Verification that $\dot{I} = T_0 \dot{S}_{\text{gen}}$ holds identically.
3. **Energy Conservation:** Closed system variations match net radiative boundary fluxes.

5 Conclusion

Sprint 019 establishes a mathematically rigorous thermodynamic foundation for the Web of Life simulation engine, ensuring that all ecological and biogeochemical processes remain physically valid.

Repository Reference: `https://github.com/pascalranoroarijaona/WebOfLife`